MOVIE BOOKING APPLICATION

Deliverables for Movie Booking Application :

User Stories

- **US_01 Registration & Login**
    - Implemented user registration with mandatory fields (First Name, Last Name, Email, Login Id, Password, Confirm Password, Contact Number).
    - Enforced uniqueness for Email and Login Id.
    - Password confirmation validation.
    - Login, logout, and password reset functionality.
    - Validation messages shown for all constraint violations.
- **US_02 View & Search Movies**
    - Users can view all released movies.
    - Search functionality implemented by movie name.
- **US_03 Book Tickets**
    - Ticket booking integrated with database table Tickets.
    - Movie Name and Theatre Name linked as foreign keys to Movie.
    - Booking captures Movie Name, Theatre Name, Number of Tickets, and Seat Numbers.
- **US_04 Admin – View & Update Tickets**
    - Admin can view booked tickets from Tickets.
    - Ticket availability calculated from Movie table.
    - Admin can update: SOLD OUT when tickets = 0 , BOOK ASAP and DELETE movie

Role-Based Authentication & Authorization

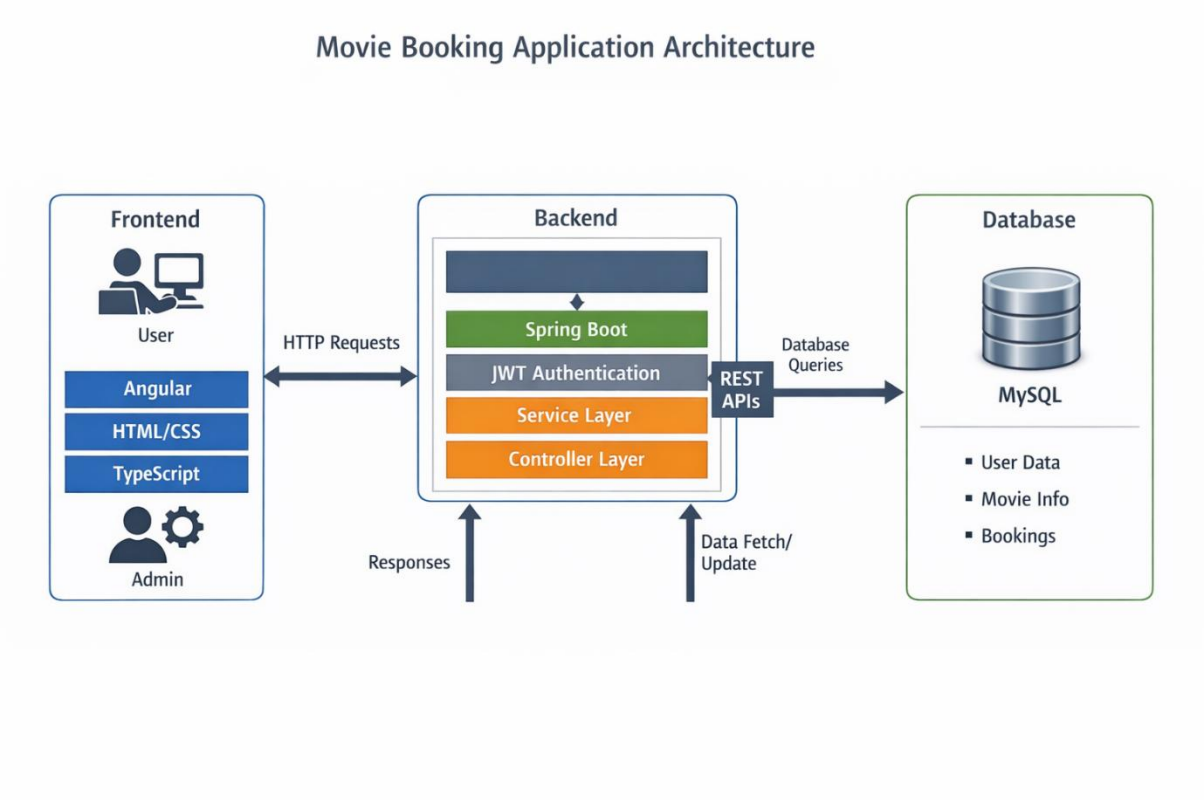
- Implemented **JWT authentication** for secure login and session management.
- Defined **two distinct roles**:
    - **USER** → can register, login, search movies, book tickets, and view their bookings.
    - **ADMIN** → can view all booked tickets, update ticket status, and manage movie/ticket availability.

Backend (Spring Boot)

- Project structured under com.moviebookingapp.*.
- Constructor-based and setter-based dependency injection applied.
- ORM implemented with:
    - JpaRepository for SQL databases (MySQL).
    - Custom queries using @Query
- Implemented caching and session management.
- Unit tests for controller, service, and repository layers with >80% coverage.
- Global exception handling for error management.
- Logging and monitoring integrated for troubleshooting.
- Simple trigger added: when tickets = 0, status auto-set to SOLD_OUT.
- Integrated **Project Lombok** to reduce boilerplate code.
- Added **mappers** for converting between entities and DTOs.

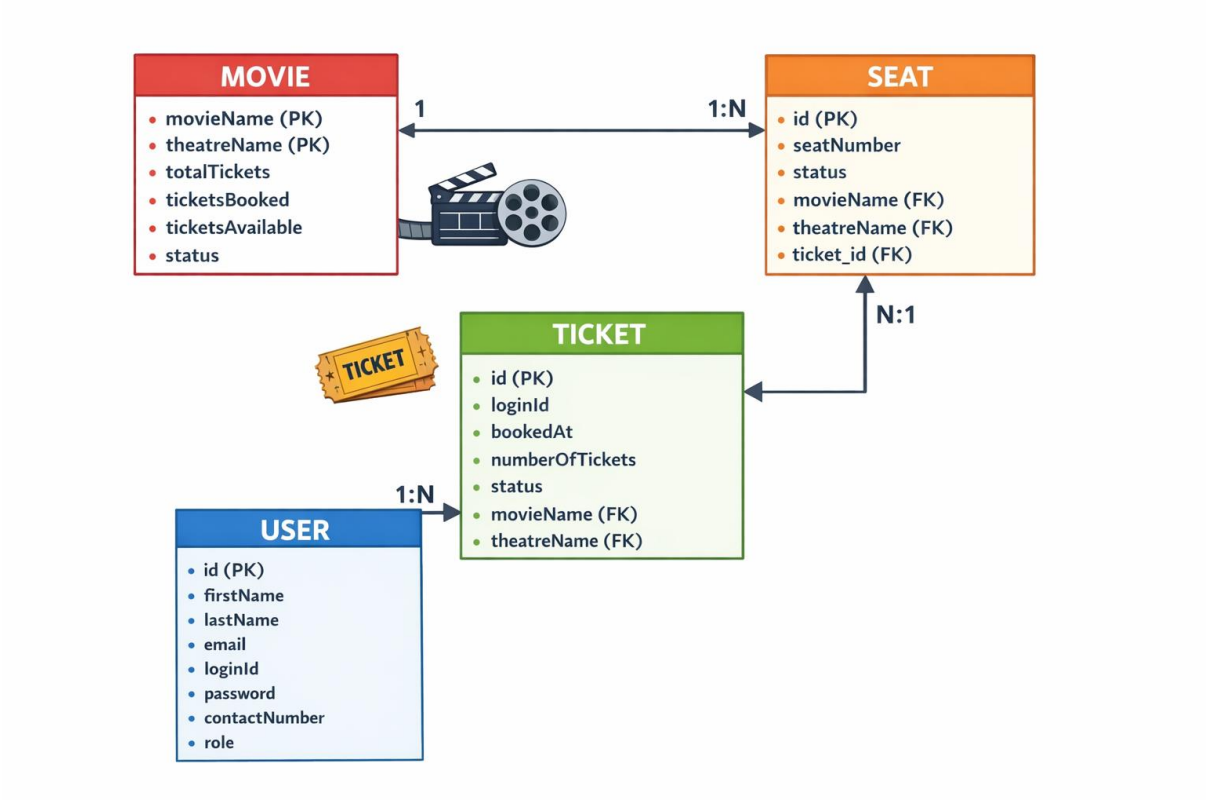
Frontend (Angular)

- Developed UI for all user stories.
- Implemented validation rules (required fields, unique checks, password match, etc.).
- Clear error/success messages with proper UI feedback.
- Consistent naming conventions and folder structures.
- Unit testing performed for components and services.



Architecture Diagram

Basic ER Diagram for Movie Booking Application



Backend Api’s

1. Registration API

Url : http://localhost:8080/api/v1.0/moviebooking/regiester

RequestPayload:

```
{
 "firstName": "shiva",
 "lastName": "rangu",
 "email": "shiva@gmail.com",
 "loginId": "123456789121",
 "password": "Shiv@1234",
 "confirmPassword":"Shiv@1234",
 "contactNumber": "9876543210"
}
```

Response :

```
{
   "success": true,
   "message": "User Created Successfully with Login Id 1234567s89121"
}
```

```
{
   "success": false,
   "message": "Email already exists"
}
```

2. Login API

Url : http://localhost:8080/api/v1.0/moviebooking/login

Request Payload:

```
{
        "loginId": "1234567890",
       "password": "Shiv@1234"
}
```

Response :

eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwicm9sZSI6IlJPTEVfVVNFUiIsImlhdCI6MTc3NDI2OTM0NSwiZXhwIjoxNzc0MjcyOTQ1fQ.HlLfy1ENCZvJU3Pc4QC3M2Ajysnv1aKKUabm_ET5tBI

3. Forgot Password :

Url : http://localhost:8080/api/v1.0/moviebooking/forgot

Request Payload

```
{
  "username":"Shiva@gmail.com",
  "newPassword":"Shiv@123456",
  "confirmPassword":"Shiv@123456"
}
```

Response :

```
{
   "success": true,
   "message": "Password Updated"
}
```

4. Reset Password

Url : http://localhost:8080/api/v1.0/moviebooking/reset

Request Payload :

```
{
   "newPassword":"Shiv@12345",
   "confirmPassword":"Shiv@12345"
}
```

Response :

```
{
   "success": false,
   "message": "New password must not be same as old password"
}
```

5. Upload Movie

Url : http://localhost:8080/api/v1.0/moviebooking/add

Request Payload :

```
{
  "movieName": "Mass",
  "theatreName": "PVR",
  "totalTickets": 20
}
```

Response

Movie Added Successfully with Seats

6. Delete Movie

Url:
http://localhost:8080/api/v1.0/moviebooking/{moviename}/delete/{theatreName}

Response

Movie Deleted Successfully

7. Update Status

Url:http://localhost:8080/api/v1.0/moviebooking/moviename/theatreName?status=BOOK_ASAP

Response :

Movie status updated to BOOK_ASAP

8. View Available Movies

Url : http://localhost:8080/api/v1.0/moviebooking/all

Response :


```
[
   {
      "movieName": "Manam",
      "theatreName": "GPR",
      "status": "BOOK_ASAP",
      "totalTickets": 10,
      "ticketsAvailable": 10,
      "ticketsBooked": 0
   },
   {
      "movieName": "shiva",
      "theatreName": "Mjr",
      "status": "AVAILABLE",
      "totalTickets": 10,
      "ticketsAvailable": 9,
      "ticketsBooked": 1
   }]
```

9. Movie Booking

Url : http://localhost:8080/api/v1.0/moviebooking/book

Request Payload :


```
{
 "movieName": "shiva",
 "theatreName": "mjr",
 "numberOfTickets": 1,
 "seatNumbers": ["A5"]
}
```


Response:

Ticket booked successfully for seats: [A5]

10. Search By Movie Name

Url : http://localhost:8080/api/v1.0/moviebooking/ movies/search/shiva

Response


```
[
   {
      "movieName": "Shiva",
      "theatreName": "Asian",
      "status": "AVAILABLE",
      "totalTickets": 10,
      "ticketsAvailable": 6,
      "ticketsBooked": 4,
      "seats": [
         {
            "seatNumber": "A1",
            "status": "BOOKED"
         },
         {
            "seatNumber": "A2",
            "status": "BOOKED"
         },
```

```
      {
        "seatNumber": "A3",
        "status": "BOOKED"
      },
      {
        "seatNumber": "A4",
        "status": "BOOKED"
      },
      {
        "seatNumber": "A5",
        "status": "AVAILABLE"
      },
      {
        "seatNumber": "A6",
        "status": "AVAILABLE"
      },
      {
        "seatNumber": "A7",
        "status": "AVAILABLE"
      },
      {
        "seatNumber": "A8",
        "status": "AVAILABLE"
      },
      {
        "seatNumber": "A9",
        "status": "AVAILABLE"
      },
      {
        "seatNumber": "A10",
        "status": "AVAILABLE"
      }
    ]
]
```

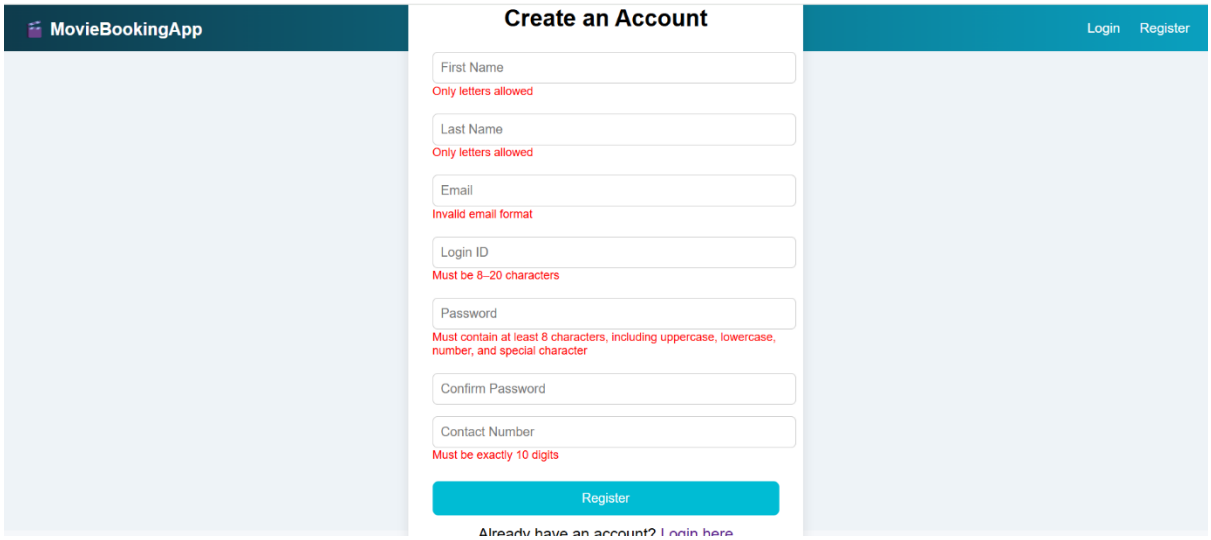
11. View Booking

Url : http://localhost:8080/api/v1.0/moviebooking/view/booking

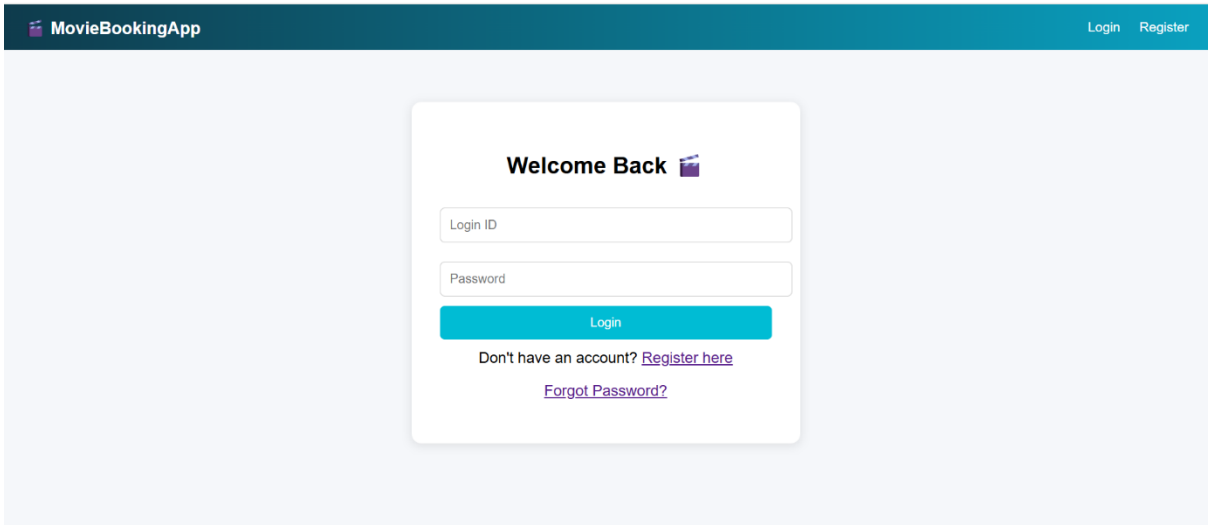
Response :


```
[
  {
    "movieName": "Shiva",
    "theatreName": "Asian",
    "numberOfTickets": 2,
    "seatNumbers": "A1, A2",
    "status": "Confirmed"
  },
  {
    "movieName": "shiva",
    "theatreName": "Mjr",
    "numberOfTickets": 1,
    "seatNumbers": "A1",
    "status": "Confirmed"
  },
  {
    "movieName": "shiva",
    "theatreName": "Mjr",
    "numberOfTickets": 1,
    "seatNumbers": "A5",
    "status": "Confirmed"
  }
]
```

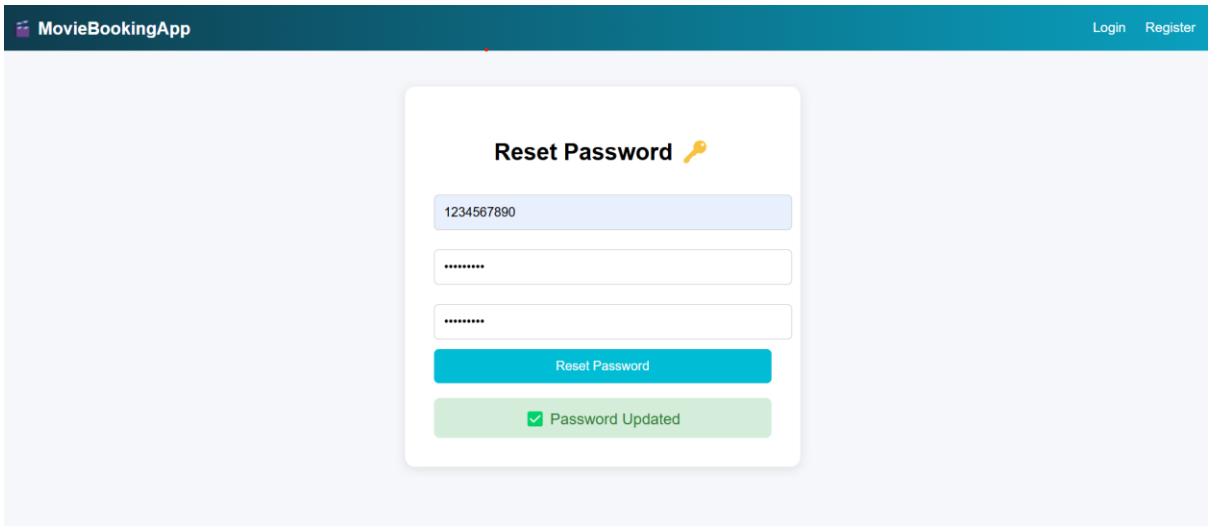
1. Register User



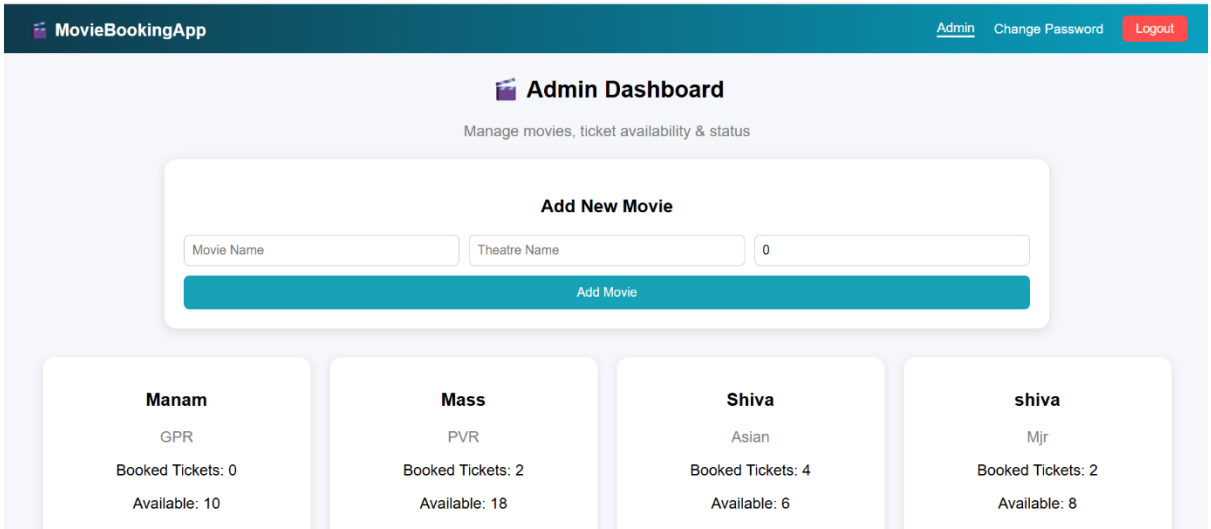
2. Login Page



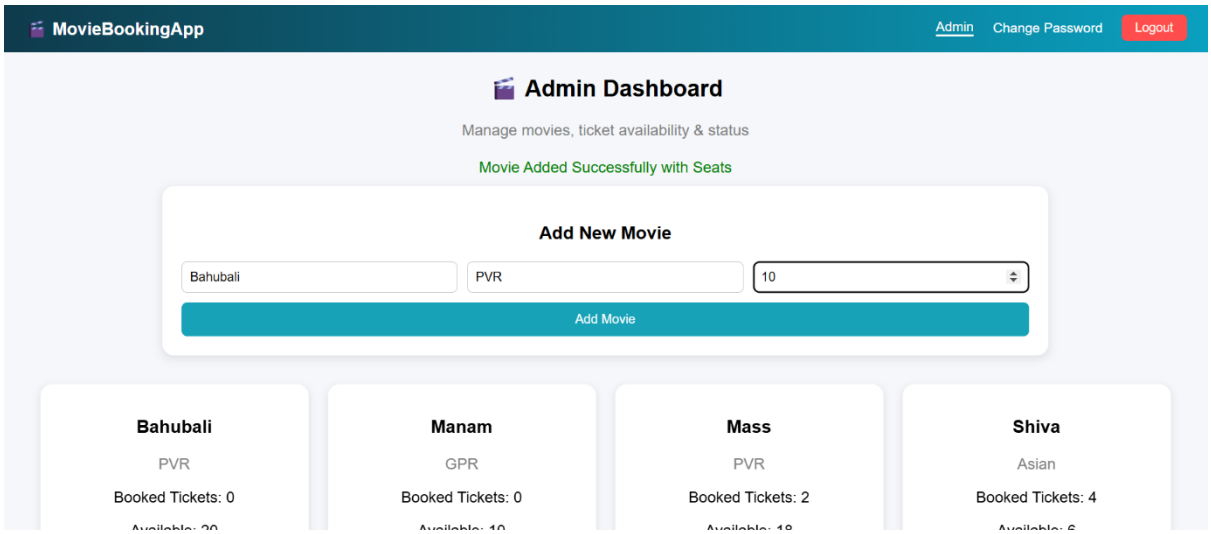
3. Reset Password when user forgot their password



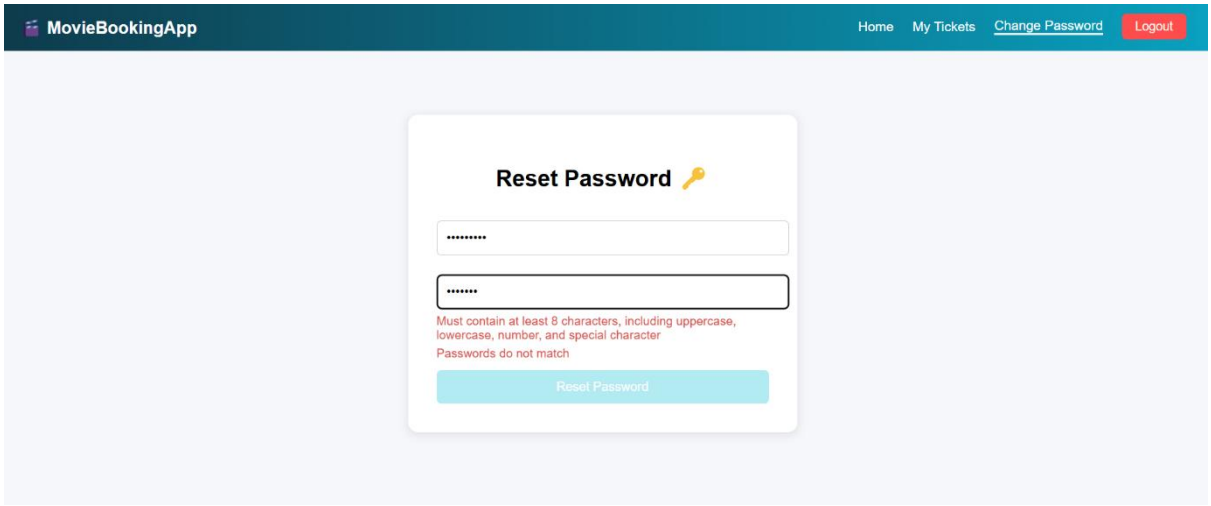
4. Admin Dashboard after login



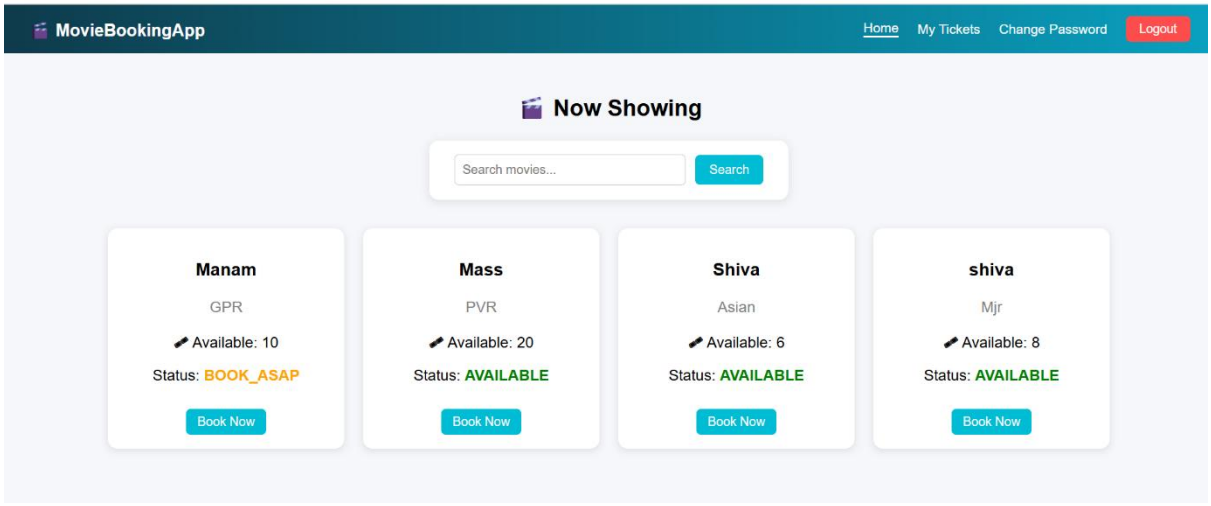
5. Admin Dashboard After movie add



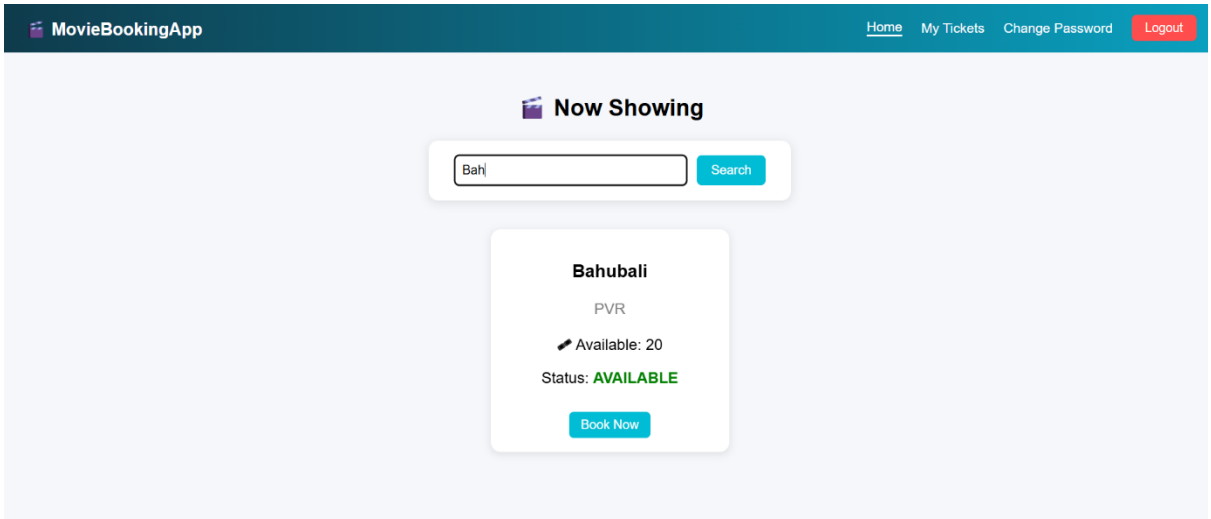
6. User can change password after login



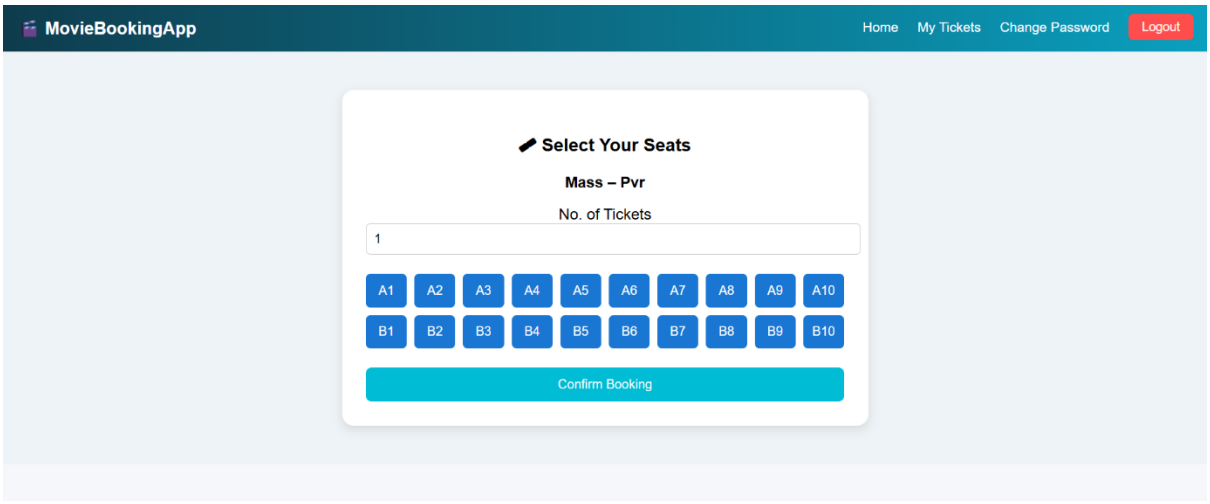
7. User Dashboard after login



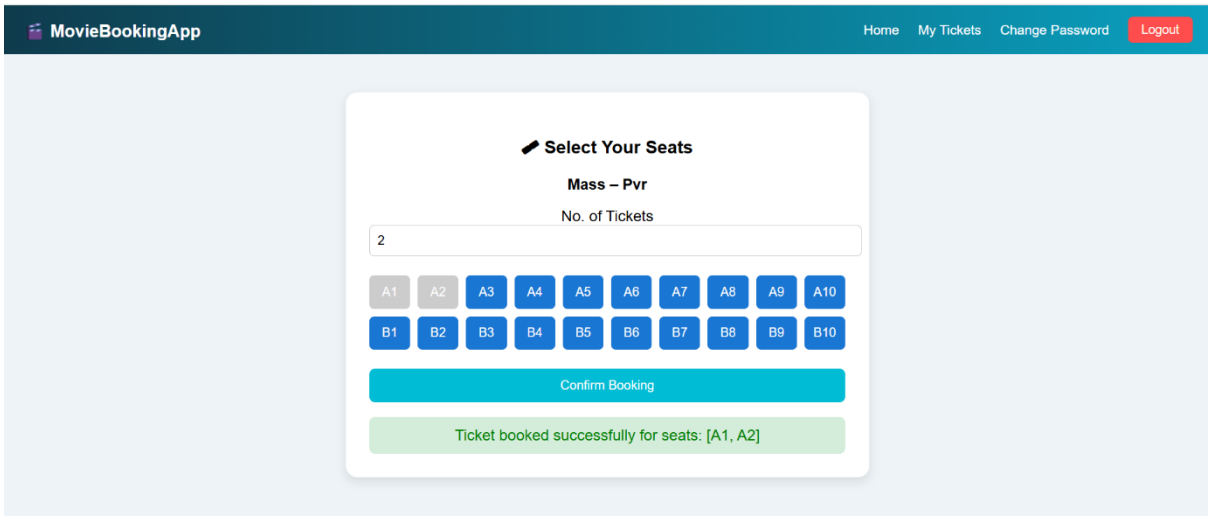
8. User can search for particular movie



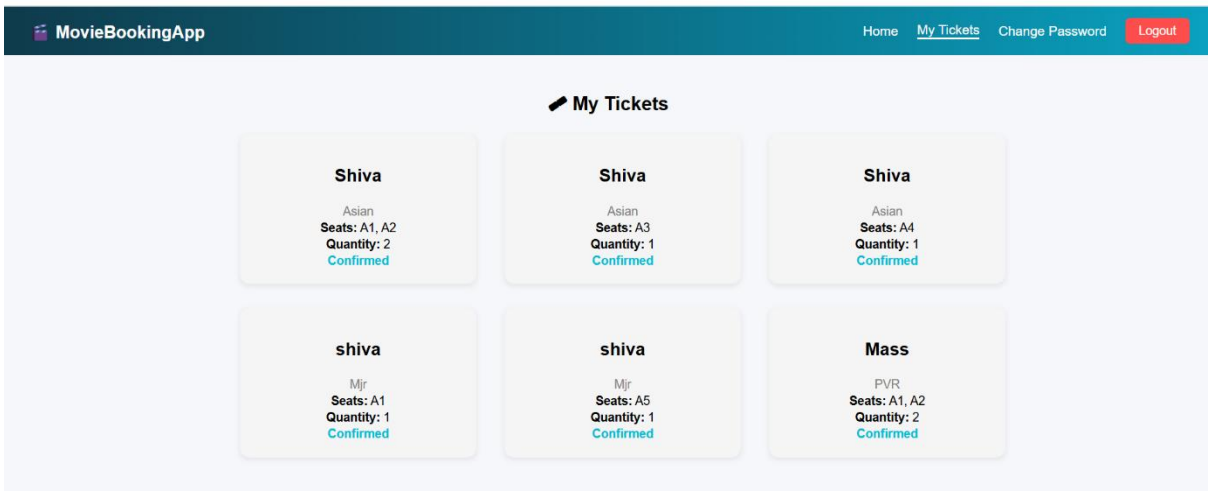
9. Seat layout after selecting movie



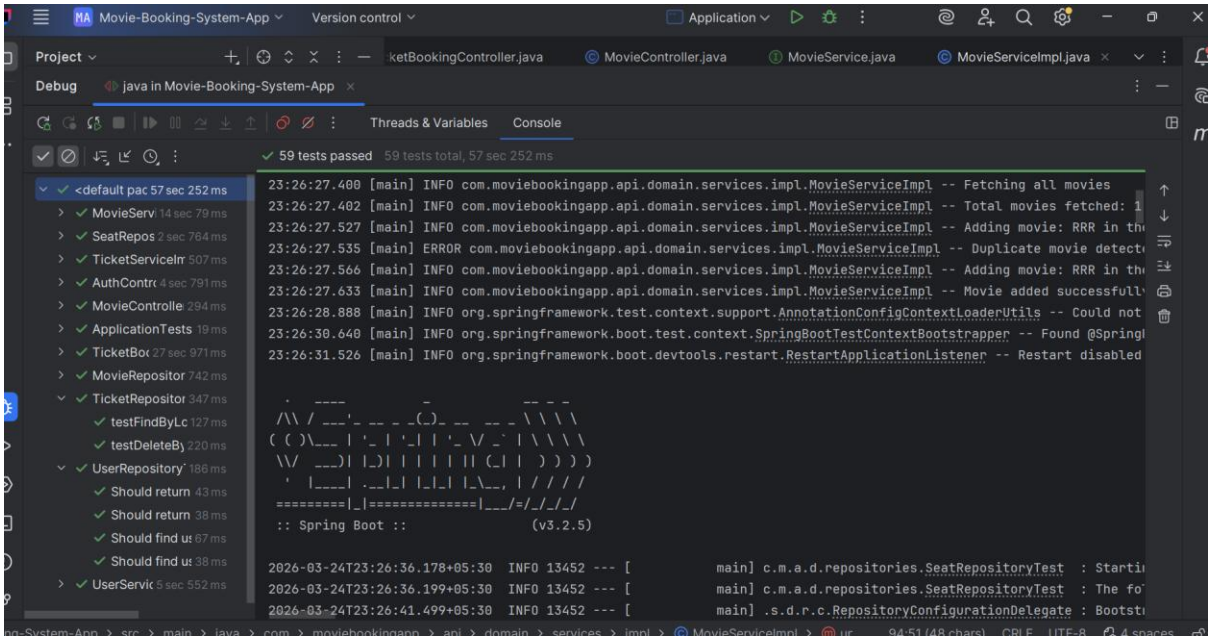
10. Seat Selection and Ticket booking

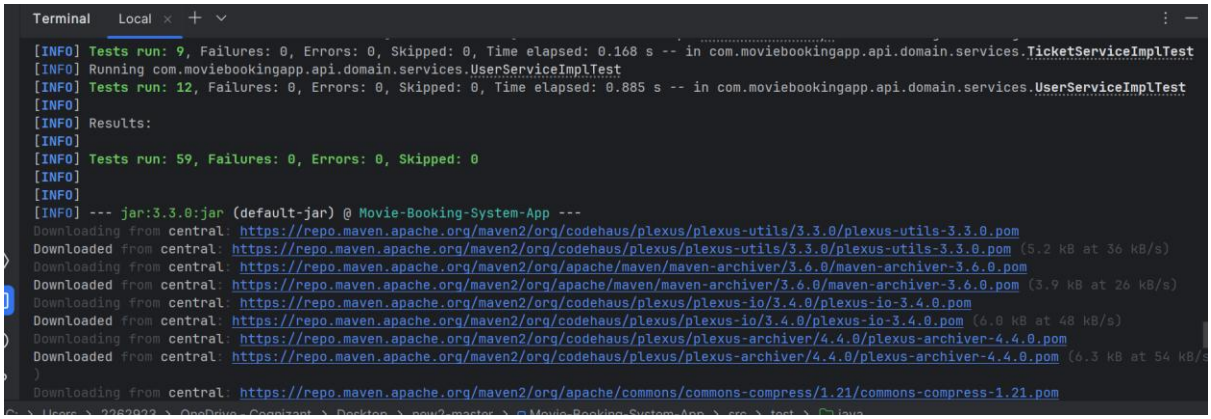


11. View Bookings



Unit Testing - Backend





Test Reports :

Movie-Booking-System-App > com.moviebookingapp.api.domain.services.impl

com.moviebookingapp.api.domain.services.impl

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
MovieServiceImpl		92%		82%	5	23	5	84	1	11	0	1
UserServiceImpl		96%		92%	2	16	1	43	1	9	0	1
TicketServiceImpl		100%		91%	1	19	0	109	0	13	0	1
Total	32 of 952	96%	6 of 49	87%	8	58	6	236	2	33	0	3

Created

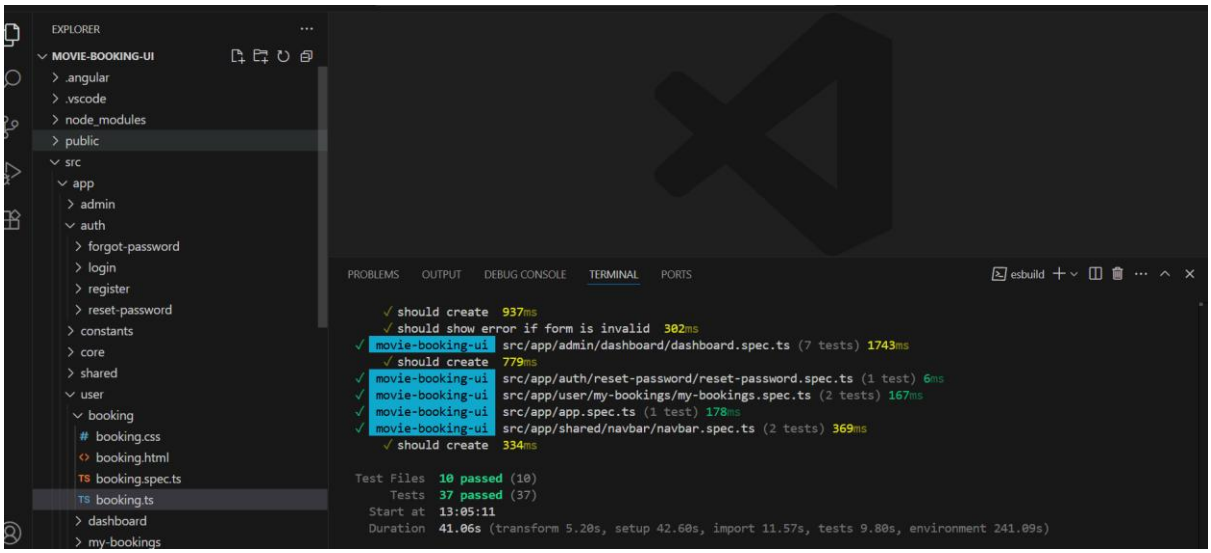
Movie-Booking-System-App > com.moviebookingapp.api.domain.controllers

com.moviebookingapp.api.domain.controllers

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
AuthController		59%		37%	4	9	11	30	0	5	0	1
TicketBookingController		100%		n/a	0	4	0	19	0	4	0	1
MovieController		100%		n/a	0	5	0	19	0	5	0	1
Total	59 of 299	80%	5 of 8	37%	4	18	11	68	0	14	0	3

Crea

Unit Testing - Frontend



The End